

Frontend: Theory Assignment 2

Complete Solutions • Deadline: 22 September 2026

1. What is CSS Box Model?

The **CSS Box Model** describes how every HTML element is treated as a rectangular box. It consists of four parts:

- **Content** – The actual text or image.
- **Padding** – Space between the content and border.
- **Border** – A line around the padding and content.
- **Margin** – Space outside the border.

Structure: Margin → Border → Padding → Content

```
Hello World

.box {
  width: 200px;
  padding: 20px;
  border: 5px solid black;
  margin: 10px;
}
```

2. What is a Media Query? Explain with a sample code.

A **media query** is a CSS feature used to make a website **responsive**. It allows different styles to be applied depending on screen size or device.

```
h1 {
  font-size: 40px;
}

@media (max-width: 600px) {
  h1 {
    font-size: 25px;
  }
}
```

Here, the heading is 40px on larger screens and 25px on screens smaller than 600px.

3. What are the features of Bootstrap?

Bootstrap is a popular front-end framework used to create responsive and attractive websites quickly.

- Responsive design – Works on mobile, tablet and desktop.
- Grid system – Arranges content in rows and columns.
- Predefined CSS classes – Provides ready-made styles.
- Buttons – Provides different button styles.
- Forms – Provides styled form elements.
- Navigation bars – Provides ready-made navigation components.
- Cards – Used to display content in boxes.
- Modals – Used for pop-up windows.

- Easy to use – Requires less custom CSS.
- Cross-browser compatibility – Works across modern browsers.

4. Write an example of basic grid structure in Bootstrap.

Bootstrap uses a **12-column grid system**. A row can be divided into different column sizes.

```
Column 1
Column 2
Column 3
```

Explanation: container creates the main container, row creates a row, and each col-4 takes 4 out of 12 columns. Three col-4 columns therefore fill the complete 12-column row.

5. What is CSS Selectors?

CSS selectors are used to select HTML elements that we want to style.

1. Element Selector: Selects elements by tag name.

```
p {
  color: blue;
}
```

2. Class Selector: Selects elements having a particular class.

```
.box {
  background-color: yellow;
}
```

3. ID Selector: Selects an element using its ID.

```
#heading {
  color: red;
}
```

4. Universal Selector: Selects all HTML elements.

```
* {
  margin: 0;
}
```

5. Group Selector: Applies the same style to multiple elements.

```
h1, h2, p {
  color: green;
}
```

6. What are CSS backgrounds? List the properties.

CSS backgrounds are used to set the background color, image and other background effects of an HTML element.

- background-color
- background-image
- background-repeat
- background-position
- background-size
- background-attachment

- **background** (shorthand property)

```
body {  
    background-color: lightblue;  
}  
  
body {  
    background-image: url("image.jpg");  
    background-repeat: no-repeat;  
    background-position: center;  
    background-size: cover;  
}
```

7. How can we overlap elements in CSS?

Elements can be overlapped using **positioning** and the **z-index** property. Commonly, position: absolute is used with a positioned parent.

```
.box1 {  
    width: 150px;  
    height: 150px;  
    background: blue;  
    position: absolute;  
    top: 50px;  
    left: 50px;  
}  
  
.box2 {  
    width: 150px;  
    height: 150px;  
    background: red;  
    position: absolute;  
    top: 100px;  
    left: 100px;  
}  
  
.box1 {  
    z-index: 1;  
}  
  
.box2 {  
    z-index: 2;  
}
```

The element with the **higher z-index** appears on top when the elements overlap.

8. What are the different positioning properties in CSS?

The CSS **position** property determines how an element is positioned on a webpage.

1. Static: The default position. The element follows the normal page flow.

```
div {  
    position: static;  
}
```

2. Relative: The element remains in the normal flow but can be moved from its original position.

```
div {  
    position: relative;  
    top: 20px;  
    left: 10px;  
}
```

3. Absolute: The element is positioned relative to its nearest positioned ancestor.

```
div {  
    position: absolute;
```

```

    top: 20px;
    left: 30px;
}

```

4. Fixed: The element stays fixed relative to the browser window, even while scrolling.

```

div {
  position: fixed;
  bottom: 10px;
  right: 10px;
}

```

5. Sticky: The element behaves normally until a scroll position is reached, then it sticks.

```

div {
  position: sticky;
  top: 0;
}

```

9. What is ECMAScript?

ECMAScript is a standard/specification that defines how scripting languages should work. **JavaScript** is the most widely used implementation of ECMAScript.

It defines features such as variables, data types, functions, objects, classes, loops, conditions, arrays and modules.

```

let name = "Saniya";
console.log(name);

```

In simple words: **ECMAScript is the standard, while JavaScript is a programming language that implements that standard.**

10. What is the difference between var, let and const in JavaScript. Explain Temporal Dead Zone.

Difference between var, let and const:

Feature	var	let	const
Scope	Function	Block	Block
Redeclaration	Allowed	Not allowed	Not allowed
Reassignment	Allowed	Allowed	Not allowed
Hoisting	Yes	Yes, but TDZ applies	Yes, but TDZ applies
Initialization	Not required immediately	Not required immediately	Required immediately

Example:

```

var a = 10;    a = 20;    // Allowed

let b = 10;    b = 20;    // Allowed

const c = 10;  // c = 20; gives an error

```

Temporal Dead Zone (TDZ): The Temporal Dead Zone is the period between entering a block and the point where a let or const variable is declared. During this period, the variable cannot be accessed.

```

console.log(x);    // ReferenceError

let x = 10;

```

Here, x is in the block's scope but cannot be accessed before its declaration. This period is called the **Temporal Dead Zone**.

Simple comparison:

```
console.log(x);  
var x = 10;    // undefined  
  
console.log(x);  
let x = 10;    // ReferenceError
```

Therefore, **var** can be accessed before its declaration due to hoisting (with the value undefined), while **let** and **const** remain in the Temporal Dead Zone until execution reaches their declaration.